

Forecasting Harga Saham Batubara IDX dengan ARIMA

Pipeline lengkap untuk mengunduh, memproses, dan memproyeksikan harga saham sektor batubara yang terdaftar di Bursa Efek Indonesia (IDX) menggunakan model ARIMA. Saham yang dianalisis meliputi **ADRO**, **PTBA**, **INDY**, dan **ITMG**, dengan cakupan forecast hingga Desember 2045.

1. Import Library

Semua dependensi yang diperlukan diimpor pada tahap ini. Library utama yang digunakan antara lain:

- **yfinance** — mengunduh data historis dari Yahoo Finance
- **statsmodels** — pemodelan ARIMA
- **sklearn** — menghitung metrik evaluasi model
- **pandas** dan **numpy** — manipulasi dan komputasi data

In [1]:

```
# -*- coding: utf-8 -*-
import sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8", errors="replace")

import json
import warnings
import numpy as np
import pandas as pd
import yfinance as yf

from datetime import datetime, date
from dateutil.relativedelta import relativedelta
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error

warnings.filterwarnings("ignore")
```

2. Konfigurasi Parameter

Seluruh parameter utama pipeline didefinisikan di sini agar mudah dimodifikasi tanpa mengubah logika inti.

Parameter ini mencakup:

- Daftar ticker saham yang akan dianalisis
- Rentang tanggal pengambilan data historis
- Tahun target akhir forecast
- Orde model ARIMA per saham
- Interval rolling re-fit model
- Profil volatilitas unik per saham

In [2]:

```
# -- Ticker Saham -----
TICKERS = {
    "ADRO": "ADRO.JK",
    "PTBA": "PTBA.JK",
    "INDY": "INDY.JK",
    "ITMG": "ITMG.JK",
}

# -- Rentang Waktu -----
# Periode download historis dibatasi Des 2024 agar forecast dimulai Jan 2025
START_DATE = "2005-01-01"
END_DATE = "2024-12-31" # forecast akan mulai Jan 2025 -> Des 2045

# Target akhir forecast
FORECAST_END_YEAR = 2045

# -- Parameter Model ARIMA per Saham -----
ARIMA_ORDERS = {
    "ADRO": (2, 1, 2),
    "PTBA": (2, 1, 2),
    "INDY": (1, 1, 1),
    "ITMG": (2, 1, 2),
}

# -- Interval Rolling Re-fit -----
# Model ARIMA di-fit ulang setiap N bulan untuk menjaga akurasi
ROLLING_REFIT_INTERVAL = 12

# -- Profil Volatilitas per Saham -----
# Setiap saham memiliki karakter naik-turun yang berbeda:
# vol = volatilitas bulanan dasar (semakin besar -> lebih fluktuatif)
# momentum = persistensi tren (semakin besar -> tren bertahan lebih lama)
# reversion = kekuatan tarik balik ke tren ARIMA
# shock_prob = peluang crash/rally mendadak per bulan
# shock_size = besarnya shock
# seed = seed unik agar pola tiap saham berbeda dan reproducible
VOL_PROFILES = {
    # ADRO: volatilitas sedang, tren menengah, sesekali shock
    "ADRO": dict(vol=0.085, momentum=0.76, reversion=0.20, shock_prob=0.07, shock_size=0.20, seed=101),
    # PTBA: lebih stabil (BUMN), volatilitas rendah, momentum pendek
    "PTBA": dict(vol=0.065, momentum=0.68, reversion=0.28, shock_prob=0.05, shock_size=0.16, seed=202),
    # INDY: paling volatile, shock lebih sering dan besar
    "INDY": dict(vol=0.12, momentum=0.80, reversion=0.15, shock_prob=0.10, shock_size=0.28, seed=303),
    # ITMG: volatilitas tinggi, tren kuat, swing besar
    "ITMG": dict(vol=0.10, momentum=0.82, reversion=0.12, shock_prob=0.08, shock_size=0.25, seed=404),
}

OUTPUT_FILE = "data_saham.json"
```

3. Fungsi-Fungsi Bantu (Helper Functions)

Seluruh fungsi yang dipakai dalam pipeline didefinisikan di sini. Masing-masing fungsi dirancang modular agar mudah diuji dan dimodifikasi secara independen.

3.1 Download & Resample Data Bulanan

Fungsi **download_monthly** mengambil data harga penutupan harian dari Yahoo Finance lalu meresamplenya ke frekuensi bulanan menggunakan rata-rata. Pendekatan ini meredam noise harian dan memberikan gambaran tren

yang lebih stabil untuk pemodelan time series.

In [3]:

```
def download_monthly(ticker_code: str, name: str) -> pd.Series:
    """
    Download data harian dari Yahoo Finance lalu resample ke bulanan
    (rata-rata harga penutupan setiap bulan).
    """
    print(f" [DL] Downloading {name} ({ticker_code}) ...")
    raw = yf.download(
        ticker_code,
        start=START_DATE,
        end=END_DATE,
        interval="1d",
        progress=False,
        auto_adjust=True,
    )

    if raw.empty:
        raise ValueError(f"Data kosong untuk {ticker_code}. Periksa koneksi/ticker.")

    # Ambil kolom Close, tangani MultiIndex jika ada
    if isinstance(raw.columns, pd.MultiIndex):
        close = raw["Close"][ticker_code]
    else:
        close = raw["Close"]

    close = close.dropna()

    # Resample ke bulanan -> rata-rata harga penutupan
    monthly = close.resample("MS").mean() # MS = Month Start
    monthly = monthly.dropna()
    monthly.name = name

    print(f"      OK {len(monthly)} bulan data "
          f"dari {monthly.index[0].date()} s/d {monthly.index[-1].date()}")
    return monthly
```

3.2 Pembagian Data Train dan Test

Data historis dibagi dengan rasio **80% train / 20% test**. Data train digunakan untuk melatih model ARIMA, sementara data test digunakan untuk mengevaluasi performa model pada data yang belum pernah dilihat sebelumnya.

In [4]:

```
def split_train_test(series: pd.Series, train_ratio: float = 0.8):
    """Split series menjadi 80% train dan 20% test."""
    n = len(series)
    split_idx = int(n * train_ratio)
    return series.iloc[:split_idx], series.iloc[split_idx:]
```

3.3 Fitting Model ARIMA

Fungsi `fit_arma` menerima data training dan parameter orde ARIMA (**p, d, q**), lalu mengembalikan model yang sudah di-fit dan siap digunakan untuk forecasting.

In [5]:

```
def fit_arma(train: pd.Series, order: tuple) -> ARIMA:
    """Fit model ARIMA pada data training."""
    model = ARIMA(train, order=order)
    return model.fit()
```

3.4 Perhitungan Metrik Evaluasi

Tiga metrik digunakan untuk mengukur performa model pada data test:

- **MAE** (Mean Absolute Error) — rata-rata selisih absolut antara nilai aktual dan prediksi
- **RMSE** (Root Mean Squared Error) — akar dari rata-rata kuadrat error, lebih sensitif terhadap outlier
- **MAPE** (Mean Absolute Percentage Error) — error dalam satuan persen, memudahkan interpretasi relatif

In [6]:

```
def compute_metrics(actual: pd.Series, predicted: pd.Series) -> dict:
    """Hitung MAE, RMSE, MAPE."""
    actual_arr = actual.values
    pred_arr = predicted.values

    mae = mean_absolute_error(actual_arr, pred_arr)
    rmse = np.sqrt(mean_squared_error(actual_arr, pred_arr))
    # MAPE -- hindari division by zero
    mask = actual_arr != 0
    mape = np.mean(np.abs((actual_arr[mask] - pred_arr[mask]) / actual_arr[mask])) * 100

    return {
        "MAE": round(float(mae), 2),
        "RMSE": round(float(rmse), 2),
        "MAPE": round(float(mape), 4),
    }
```

3.5 Rolling Forecast hingga 2045

Fungsi `rolling_forecast_to_2045` menghasilkan proyeksi harga saham hingga Desember 2045 dengan dua lapisan:

- **Lapisan 1 — Tren Jangka Panjang (ARIMA):** Model ARIMA di-fit ulang setiap *refit_interval* bulan. Hasil forecast ARIMA berfungsi sebagai anchor atau target mean-reversion.
- **Lapisan 2 — Simulasi Volatilitas Realistik:** Di atas tren ARIMA, ditambahkan simulasi berbasis proses Ornstein-Uhlenbeck dalam log-space. Pendekatan ini memastikan ada momentum, mean-reversion, volatilitas bulanan yang realistis, serta sesekali muncul shock besar (+20-28%) yang menggambarkan crash/rally batubara.

In [7]:

```
def rolling_forecast_to_2045(
    full_series: pd.Series,
    order: tuple,
    refit_interval: int = ROLLING_REFIT_INTERVAL,
    vol: float = 0.09,
    momentum: float = 0.78,
    reversion: float = 0.18,
    shock_prob: float = 0.08,
    shock_size: float = 0.22,
    seed: int = 42,
) -> pd.Series:
    """
    Forecast rolling hingga Desember 2045 dengan simulasi volatilitas realistis.
    1. ARIMA di-refit setiap refit_interval bulan -> tren jangka panjang sebagai anchor.
    2. Harga disimulasikan dengan mean-reverting random walk (Ornstein-Uhlenbeck).
    3. Sesekali ada shock besar (+/-22%) seperti crash/rally batubara.
    """
    rng = np.random.default_rng(seed)
    last_date = full_series.index[-1]
    target = pd.Timestamp(f"{FORECAST_END_YEAR}-12-01")

    n_forecast_total = (
        (target.year - last_date.year) * 12
        + (target.month - last_date.month)
    )
    if n_forecast_total <= 0:
        print(f"      [WARN] Data historis sudah melampaui 2045.")
        return pd.Series(dtype=float)

    print(f"      [FORECAST] Rolling forecast {n_forecast_total} bulan "
          f"(vol={int(vol*100)}%/bln, momentum={momentum}) ...")

    # -- Kumpulkan seluruh tren ARIMA dulu (tanpa noise) -----
    arima_trend = []
    trend_dates = []
    living_series = full_series.copy().astype(float)
    month_offset = 0
```

In [7b]:

```
while month_offset < n_forecast_total:
    batch_size = min(refit_interval, n_forecast_total - month_offset)
    try:
        fitted = ARIMA(living_series, order=order).fit()
        fc = fitted.forecast(steps=batch_size)
    except Exception as e:
        print(f"      [WARN] ARIMA offset {month_offset}: {e}. Pakai konstanta.")
        last_v = float(living_series.iloc[-1])
        fc = pd.Series([last_v] * batch_size)

    for i, val in enumerate(fc):
        new_date = living_series.index[-1] + relativedelta(months=1 + i)
        trend_dates.append(new_date)
        arima_trend.append(max(float(val), 1.0))

    new_idx = pd.DatetimeIndex([
        living_series.index[-1] + relativedelta(months=i + 1)
        for i in range(batch_size)
    ])
    living_series = pd.concat([
        living_series,
        pd.Series(arima_trend[-batch_size:], index=new_idx)
    ])
    month_offset += batch_size

# -- Simulasi harga realistis di atas tren ARIMA -----
forecast_values = []
deviation = 0.0

for i, (trend_val, d) in enumerate(zip(arima_trend, trend_dates)):
    z = rng.standard_normal()
    if rng.random() < shock_prob:
        z += rng.choice([-1, 1]) * rng.uniform(1.5, shock_size / 0.09)

    deviation = (momentum - reversion) * deviation + vol * z
    deviation = np.clip(deviation, -0.80, 0.80)

    price = trend_val * np.exp(deviation)
    price = max(price, 1.0)
    forecast_values.append(round(float(price), 2))

forecast_series = pd.Series(forecast_values, index=pd.DatetimeIndex(trend_dates))
return forecast_series
```

3.6 Walk-Forward Prediction pada Data Test

Untuk mengevaluasi model secara jujur, digunakan metode **walk-forward validation**: model di-fit ulang setiap satu langkah waktu menggunakan semua data yang tersedia hingga titik tersebut, lalu memprediksi satu langkah ke depan. Ini mencerminkan kondisi nyata di mana model hanya boleh menggunakan informasi masa lalu.

In [8]:

```
def predict_test(train: pd.Series, test: pd.Series, order: tuple) -> pd.Series:
    """
    Prediksi satu langkah ke depan secara rolling untuk data test
    (walk-forward validation).
    """
    predictions = []
    history = list(train.values)

    for val in test.values:
        try:
            model = ARIMA(history, order=order)
            result = model.fit()
            yhat = float(result.forecast(steps=1).iloc[0])
        except Exception:
            yhat = history[-1] # fallback: nilai sebelumnya

        predictions.append(yhat)
        history.append(val) # update history dengan nilai aktual

    return pd.Series(predictions, index=test.index)
```

3.7 Konversi Series ke Format JSON

Fungsi utilitas untuk mengonversi **pd.Series** bertipe datetime-index menjadi list of dictionary **{date, value}** yang kompatibel dengan format JSON dan siap dikonsumsi oleh dashboard HTML.

In [9]:

```
def series_to_records(series: pd.Series) -> list:
    """Konversi pd.Series ke list of {date, value} untuk JSON."""
    return [
        {
            "date": idx.strftime("%Y-%m-%d"),
            "value": round(float(val), 2) if not np.isnan(val) else None,
        }
        for idx, val in series.items()
    ]
```

4. Pipeline Utama

Fungsi **main** mengorkestrasi seluruh tahapan pipeline secara berurutan untuk setiap saham:

- **1. Download & resample** — ambil data harian, konversi ke rata-rata bulanan
- **2. Train/test split** — bagi 80% training, 20% testing
- **3. Walk-forward validation** — prediksi rolling pada data test
- **4. Evaluasi model** — hitung MAE, RMSE, MAPE
- **5. Rolling forecast** — proyeksi ke Desember 2045 dengan volatilitas per saham
- **6. Ekspor JSON** — simpan semua hasil ke data_saham.json

In [10]:

```
def main():
    print("=" * 60)
    print(" ARIMA Forecast Saham Batubara IDX -> data_saham.json")
    print("=" * 60)
    print(f" Ticker   : {' ', ' '.join(TICKERS.keys())}")
    print(f" Periode  : {START_DATE} ~ {END_DATE}")
    print(f" Target   : Forecast hingga Desember {FORECAST_END_YEAR}")
    print("=" * 60)

    output = {
        "meta": {
            "generated_at": datetime.now().isoformat(),
            "forecast_end": f"{FORECAST_END_YEAR}-12-01",
            "tickers":      list(TICKERS.keys()),
            "description": (
                "Data bulanan rata-rata harga penutupan saham batubara IDX. "
                f"ARIMA train/test split 80/20. Rolling forecast hingga {FORECAST_END_YEAR} "
                f"dengan re-fit setiap {ROLLING_REFIT_INTERVAL} bulan. "
                f"Simulasi volatilitas per saham: "
                f"ADRO={int(VOL_PROFILES['ADRO']['vol']*100)}%, "
                f"PTBA={int(VOL_PROFILES['PTBA']['vol']*100)}%, "
                f"INDY={int(VOL_PROFILES['INDY']['vol']*100)}%, "
                f"ITMG={int(VOL_PROFILES['ITMG']['vol']*100)}%."
            ),
        },
        "stocks": {},
    }

    for name, ticker_code in TICKERS.items():
        print(f"\n{'-'*50}")
        print(f"    [STOCK] Memproses: {name} ({ticker_code})")
        print(f"{'-'*50}")

        try:
            # 1. Download & resample bulanan
            monthly = download_monthly(ticker_code, name)

            # 2. Train / Test split
            train, test = split_train_test(monthly, train_ratio=0.8)
            print(f"    Train: {len(train)} bln | Test: {len(test)} bln")

            # 3. Walk-forward prediction pada test set
            print(f"    [CALC] Walk-forward prediction pada test set ...")
            order = ARIMA_ORDERS[name]
            test_preds = predict_test(train, test, order)

            # 4. Hitung metrik
            metrics = compute_metrics(test, test_preds)
            print(f"    METRICS: MAE={metrics['MAE']:.0f} "
                  f"RMSE={metrics['RMSE']:.0f} MAPE={metrics['MAPE']:.2f}%")
```


In [10b]:

```
# 5. Rolling forecast ke 2045 dengan profil volatilitas unik per saham
vp = VOL_PROFILES.get(name, {})
forecast_series = rolling_forecast_to_2045(monthly, order, **vp)

# 6. Susun output per saham
output["stocks"][name] = {
    "ticker":    ticker_code,
    "order":     list(order),
    "metrics":   metrics,
    "historical": series_to_records(monthly),
    "test_actual": series_to_records(test),
    "test_predicted": series_to_records(test_preds),
    "forecast":   series_to_records(forecast_series),
}

except Exception as e:
    print(f"  ERROR pada {name}: {e}")
    output["stocks"][name] = {"error": str(e)}

# 7. Simpan ke JSON
print(f"\n{' '*60}")
print(f"  [SAVE] Menyimpan hasil ke {OUTPUT_FILE} ...")
with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
    json.dump(output, f, ensure_ascii=False, indent=2)

print(f"  [OK] Selesai! File tersimpan: {OUTPUT_FILE}")
print(f"{' '*60}")

# Ringkasan metrik
print("  RINGKASAN METRIK TESTING")
print(f"  {'Saham':<8} {'MAE':>12} {'RMSE':>12} {'MAPE':>10}")
print(f"  {'-'*46}")
for name, data in output["stocks"].items():
    if "metrics" in data:
        m = data["metrics"]
        print(f"    {name:<8} {m['MAE']:>12, .2f} "
              f"    {m['RMSE']:>12, .2f} {m['MAPE']:>9.2f}%")
    else:
        print(f"    {name:<8} {'ERROR':>12}")
print()
```

5. Eksekusi Pipeline

Jalankan sel di bawah ini untuk memulai seluruh proses: download data, training model, evaluasi, forecasting, dan ekspor hasil ke file **data_saham.json**. Estimasi waktu bervariasi tergantung kecepatan koneksi dan spesifikasi mesin, umumnya antara **5-15 menit** karena proses walk-forward validation yang intensif.

In [11]:

```
if __name__ == "__main__":
    main()
```